In **Imbalanced Data**, the model may ignore the minority class because it's too small.
Example: 95% healthy, 5% sick → a model could predict "healthy" always and still get 95% accuracy.

⚡ Assigning Higher Weights

- In **Logistic Regression (or other models)**, the **loss function** measures how wrong the model is.
- Normally, each mistake counts equally.
- With **class weights**, mistakes on the minority class are made **more costly** than mistakes on the majority class.

✅ Example

Suppose we set:

- Weight for majority class (healthy) = 1
- Weight for minority class (sick) = 5

Now:

- If the model misclassifies 1 sick person → penalty = 5
- If it misclassifies 1 healthy person → penalty = 1

So the model learns:
👉 "It's worse to miss a sick person than to mislabel a healthy one."

This makes the model **pay more attention to the minority class**.

👉 In short: **Class weights increase the importance of minority class errors, forcing the model to treat them more seriously.**

🔹 **Standard Logistic Regression Loss (Binary Cross-Entropy):**

$$Loss = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \cdot \log(p_i) + (1-y_i)\cdot \log(1-p_i)\right]$$

- y_i = actual label (0 or 1)
- p_i = predicted probability of class 1

Here, every sample contributes **equally**.

🔹 **With Class Weights:**

$$Loss = -\frac{1}{N}\sum_{i=1}^{N}\left[w_1 \cdot y_i \cdot \log(p_i) + w_0 \cdot (1-y_i)\cdot \log(1-p_i)\right]$$

- w_1 = weight for minority class (positive class)
- w_0 = weight for majority class (negative class)

✅ **Example**

If dataset = 95% class 0, 5% class 1 → we might set:

- $w_0 = 1$
- $w_1 = 19$ (so the small class gets ~19x importance)

This way, misclassifying class 1 is punished much more.

👉 In short: **Class weights modify the loss function so that mistakes on minority samples "cost more," forcing the model to learn them.**

How to do it in **scikit-learn (sklearn)** with Logistic Regression.

✅ Option 1: Use `class_weight="balanced"`

This makes sklearn automatically assign weights **inversely proportional to class frequencies**.

python Copy code

```python
from sklearn.linear_model import LogisticRegression

# Logistic Regression with automatic balancing
model = LogisticRegression(class_weight="balanced")
model.fit(X_train, y_train)
```

So if your data is 95% class 0 and 5% class 1 → sklearn will give class 1 about **19x more weight**.

✅ Option 2: Set Custom Weights

You can manually set weights if you want more control.

python Copy code

```python
from sklearn.linear_model import LogisticRegression

# Example: Give class 0 weight = 1, class 1 weight = 10
weights = {0: 1, 1: 10}

model = LogisticRegression(class_weight=weights)
model.fit(X_train, y_train)
```